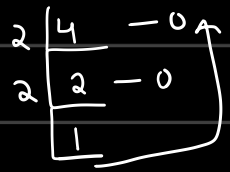
BIT MANIPULATION

→ Binary Number System :-

Decimal to Binary (4)



$(100)_2 = (4)_{10}$

Binary to Decimal (100)

$100 = 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$

$= 4$

$(100)_2 = (4)_{10}$

→ Bitwise operators :-

i) Binary AND (&) –

Rules :

0 & 0 = 0

0 & 1 = 0

1 & 0 = 0

1 & 1 = 1

eg :- 5 & 6 ⇒ A = 101 B = 110

$$\begin{array}{r} 101 \\ \& \ 110 \\ \hline 100 \end{array} \Rightarrow A \ \& \ B = 4$$

ii) Binary OR (|) –

Rules :

0 | 0 = 0

1 | 1 = 1

$1 \mid 0 = 1$

$1 \mid 1 = 1$

eg:- $5 \mid 6 = ?$

$A = (5)_{10} = (101)_2$ $\quad B = (6)_{10} = (110)_2$

$$\begin{array}{r} 101 \\ 110 \\ \hline 111 \end{array} \Rightarrow A \mid B = 7$$

iii) Binary XOR ($\wedge$) -

Rules :

$0 \wedge 0 = 0$

$0 \wedge 1 = 1$

$1 \wedge 0 = 1$

$1 \wedge 1 = 0$

eg:- $5 \wedge 6 = ?$

$$\begin{array}{r} 101 \\ \wedge\ 110 \\ \hline 011 \end{array} \Rightarrow A \wedge B = 3$$

★ iv) Binary One's complement ($\sim$): ← NOT

Rules :

$\sim 0 = 1$

$\sim 1 = 0$

eg:- $\sim 5 \Rightarrow \sim 101 = 010 = (2)_{10}$

But this won't be the answer in computer

Most significant bit (MSD) ↑ ↗ least significant bit

In computer 5 is stored as ⇒ 00000101

if MSD = 0 ⇒ no. is +ve

MSD = 1 ⇒ no. is -ve

⇓ ~5

11111010 → no. becomes -ve

1's complement ↓ to find value of this

2's complement

-(00000101)

↓ +1

-(00000110)

↓

-6

~5 = -6

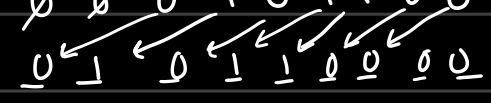
In short, complement of x is -(x+1).

v) Binary Left Shift (<<) -

Rules:

Let a = 000101100, b = 2

a << b(2)



eg :- 5 << 2 ⇒ a = 00000101, b = 2

⇒ 000101100 = a<<b

⇒ 20 = a<<b

Formula ⇒ $\boxed{a << b = a \times 2^b}$

vi) Binary Right Shift (>>) -

Rules:

a >> b

a = 0 01001 00

b = 2

0 0 1 0 0 1 0 0

0 0 0 0 1 0 0 1

eg :- $6 >> 1 \Rightarrow a = 000110 \;,\; b = 1$

$\Rightarrow 000011 = a >> b$

$\Rightarrow 3 = a >> b$

Formula $\Rightarrow$ $\boxed{a >> b = a / 2^b}$

1) Check if number is odd or even.

Sol :- Approach:

0 = 0 0 0

1 = 0 0 1

2 = 0 1 0

3 = 0 1 1

4 = 1 0 0

5 = 1 0 1

→ we can notice that the number is odd only when LSD = 1

↓ to find this

we can (n & 1) ⇒ It will give 1 when LSD = 1, else it gives 0.

→ Operations :-

i) Get i^{th} bit:

n = 1 0 0 0 1 0 0 1 1 0 0

. . . . . 3 2 1 0

i = 2 ⇒ i^{th} bit = 1

generalization:

eg :- 0 0 0 0 1 1 1 1 $\quad$ i = 2

& (0 0 0 0 0 1 0 0) → 1 << 2 (1 << i)

0 0 0 0 0 1 0 0 $\neq 0 \rightarrow i^{th}$ bit 1

n & (1<<i) → 0 (0)
→ ≠0 (1)

ii) Set i^{th} bit:

set i=2 to 1

0 0 0 0 1 0 1 0

↓

0 0 0 0 1 1 1 0

generalization:

eg:- 0 0 0 0 1 0 0 0 i=2

1 (0 0 0 0 0 1 0 0) → 1 << 2 (1<<i)

0 0 0 0 1 1 0 0

n | (1<<i)

iii) Clear i^{th} bit:

clear i=1

1 0 1 0

↓

1 0 0 0

generalization:

eg:- i=1 → (1<<i) → ~(0 0 1 0)

↓

(1 1 0 1)

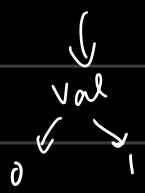
1 0 1 0

& (1 1 0 1)

1 0 0 0

n & (~(1<<i))

iv) Update i^{th} bit:




```
if (val == 0)
    return clear_ith_bit(n, i)
else
    return set_ith_bit(n, i);
```

(OR)

First clear the ith bit, then

if nB=0

nB << i

0 << i | n

⇒ n

else nB=1

(nB << i) | n

↓

ith bit set

v) Clear last i bits:

n = 1111, i = 2

1 1 1 1

↓

1 1 0 0

~0 = 11111111 = -1

↓ i bits <<

11 111100

generalization:

```
  1 0 1 1
& (1 1 0 0)  ⟶  ((~0) << i)
-----------        or
  1 0 0 0        ((-1) << i)
```

num & ((~0) << i)

vi) Clear range of bits:

$n = 100111010011$, $i=2, j=7$

1 0 0 1 1 1 0 1 0 0 1 1

7 2

↓

1 0 0 1 0 0 0 0 0 0 1 1

generalization:

1 0 0 1 1 1 0 1 0 0 1 1

& (1 1 1 1 0 0 0 0 0 1 1)

1 0 0 1 0 0 0 0 0 0 1 1

$a = (\sim 0) << (j+1)$

$01 = (1)_{10} = 2^1 - 1$

$011 = (3)_{10} = 2^2 - 1$

$0111 = (7)_{10} = 2^3 - 1$

$01111 = (15)_{10} = 2^4 - 1$

$2^x = 1 << x$

$b = (1 << i) - 1$

a | b

$a = 11110\ 0000000$ (→ j+1)

$b = 0000000000\ 11$ (i−1)

num & (a | b)

where, $a = (\sim 0) << j+1$

$b = (1 << i) - 1$

$b = 0000 \ldots 11$

$\sim ((-1) << i)$

2) Check is a number is power of 2 or not?

4 → 100

& 3 → 011 → 0

8 → 1000

& 7 → 0111 → 0

16 → 10000

15 → 01111 → 0

n & (n−1) = 0

* (gogle)

3) Count set bits in a Number.

Approach → 10 → 1010

↓

no. of set bits = 2

$n = \log_2 n + 1$

Eg:- $16 = \underbrace{10000}_{5}$

$= \log_2 16 + 1 = 4 + 1$

$= 5$

⇒ i) n = 1010 count = 0

n >> 1 (LSB)

ii) n = 0101 count = 1

(LSB)

n >> 1

iii) n = 0010 count = 1

(LSB)

n >> 1

iv) n = 0001 count = 2

(LSB)

n >> 1

v) n = 0000

(LSB)

↓

count = 2

O(log n)

→ Fast Exponentiation:

$a^5 = a^{101}$ (a^4 a^2 a; 1 0 1)

$(1 \times a^4) \times (1) \times (1 \times a) = a^5$

$3^5 \rightarrow 3^{101}$

① a = 3; ans = 1

ans = 3

$a = a^2 = 9$

② a = 9; ans = 3

ans = 3 × 1

$a = a^2 = 81\ (a^4)$

③ a = 81; ans = 3

ans = 3 × 81 = 243

$a = a^2 = (81)^2$

only 3 iterations instead of 5

So we can solve in O(log n)

→ Modular Exponentiation:

$$(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$$

Proof :- Let $a = qm + r$ where, q = quotient, m = divisor, r = remainder

Similarly, $b = pm + s$

Now, $a \times b = (qm + r)(pm + s)$

$= qpm^2 + qms + rpm + rs$

$= m(qpm + qs + rp) + rs$

We see that, everything except rs is divisible by m.

So when we take modulo, those terms disappear.

So,

$$(a \times b) \bmod m = (r \times s) \bmod m$$

But,

$$r = a \bmod m$$

$$s = b \bmod m$$

$$\Rightarrow (a \times b) \bmod m = ((a \bmod m)(b \bmod m)) \bmod m$$

eg:- 5^{1010} → $ans = 1, n = 5, pow = 10, mod = 13$

$3^{10} \bmod 13$

① 1010
bit = 0
ans = ans
$n = (5 \times 5) \bmod 13 = 12$

② 0101
bit = 1
$ans = (1 \times 12) \bmod 13$
$n = (12 \times 12) \bmod 13 = 1$

③ 0010
bit = 0
ans = ans
$n = (1 \times 1) \bmod 13 = 1$

④ 0001
bit = 1
$ans = (12 \times 1) \bmod 13 = 12$
$n = (1 \times 1) \bmod 13 = 1$

Final → 12